

On-Chip Toggle-Counter Fingerprinting as a Runtime Power Side-Channel Proxy for Hardware Trojan Detection in FPGA-Based AES Cryptographic Cores

Sarath Srinivasan, Vaasudev Enamandram, Mukund Varshney, and Abhishek Kumar

Abstract—Hardware Trojans can remain dormant during conventional verification and activate only under rare conditions, making them difficult to detect through functional testing alone. This paper presents an on-chip toggle-counter fingerprinting framework for real-time Hardware Trojan detection in FPGA-based AES cryptographic cores. Instead of relying on external power probes or offline trace acquisition, the proposed method embeds digital activity sensors inside the AES-128 core and uses internal state, key, and control-path switching activity as a side-channel proxy for power. The sensors compute per-transaction Hamming-distance activity using XOR, popcount, and accumulation logic. A Trojan-free AES baseline is used to derive statistical thresholds, and the framework is evaluated on two Trojan families: a Performance Degradation Trojan with two variants and an Information Leakage Trojan inspired by the Trust-Hub AES-T1000-style plaintext-triggered key-leakage Trojan. FPGA experiments on a Xilinx Artix-7 platform show that the control-path fingerprint is highly discriminative: the clean AES baseline remains fixed at 21, while Trojan cases produce threshold-crossing control activity up to 39, 57, and 24, respectively. The sensorized AES implementation incurs 22.01% LUT overhead, 22.83% FF overhead, and only 2.90% power overhead while maintaining timing closure.

Index Terms—Hardware Trojan, FPGA security, AES, side-channel analysis, runtime monitoring, toggle counter, activity fingerprinting, power side-channel proxy, cryptographic hardware.

I. INTRODUCTION

The globalization of the semiconductor and FPGA design ecosystem has introduced serious hardware trust concerns. Modern cryptographic systems often depend on third-party intellectual property, automated EDA tools, external libraries, and distributed design or fabrication flows. While this model reduces cost and development time, it also creates opportunities for malicious modifications known as Hardware Trojans [1], [11].

Hardware Trojans are particularly dangerous in cryptographic cores because they may leak secret keys, corrupt ciphertexts, reduce reliability, or degrade performance. A Trojan commonly consists of a trigger and a payload. The trigger activates the malicious logic under a rare condition, while the payload performs the attack. Since the trigger condition may be extremely unlikely during normal testing, conventional functional validation may not expose the Trojan [1], [6].

Side-channel analysis has therefore become an important direction for Trojan detection. Prior works have investigated power, current, delay, electromagnetic, thermal,

and ring-oscillator-based signatures to identify Trojan-induced deviations [2]–[5], [7]. However, many side-channel methods require external measurement equipment, oscilloscope-style trace collection, trace alignment, offline preprocessing, or complex statistical calibration. These requirements can limit practical deployment in lightweight FPGA-based cryptographic systems.

This paper proposes an alternative runtime monitoring approach: instead of measuring only external physical traces, the AES core is instrumented with embedded toggle-counter sensors. These sensors directly measure internal switching activity and generate activity fingerprints for each encryption transaction. Since dynamic switching activity is closely related to power consumption, the proposed activity counters act as digital side-channel proxies.

The central novelty of this work is *on-chip toggle-counter fingerprinting* for real-time Hardware Trojan detection. The framework measures AES state, key, and control-path activity during normal operation and uses baseline-trained thresholds to detect runtime deviations. The proposed method is evaluated on an FPGA AES platform using a Performance Degradation Trojan and an Information Leakage Trojan.

The main contributions of this work are as follows:

- An FPGA-based AES-128 Trojan detection platform with UART-driven plaintext injection, ciphertext validation, and activity-feature extraction.
- On-chip toggle-counter sensors that compute per-transaction AES state, key, control, and total switching activity using XOR-popcount accumulation.
- A baseline-trained statistical detection method using Trojan-free AES activity fingerprints.
- Evaluation using two Trojan families: a Performance Degradation Trojan with two cases and an Information Leakage Trojan with AES key-leakage payload.
- Post-implementation overhead analysis showing low power overhead and maintained timing closure on a Xilinx Artix-7 FPGA.

II. BACKGROUND AND RELATED WORK

A. Hardware Trojan Threat Model

A Hardware Trojan is an intentional malicious modification to a circuit. Trojans may be inserted during design, IP integration, EDA processing, fabrication, packaging, or deployment [1], [11]. Their payloads may leak information, alter functionality, degrade performance, or reduce reliability.

The authors are with the Department of Electrical Engineering, Indian Institute of Technology Ropar, Punjab, India. (e-mail: {2023eeb1242, 2023eeb1249, 2023eeb1226}@iitrpr.ac.in; akumar@iitrpr.ac.in).

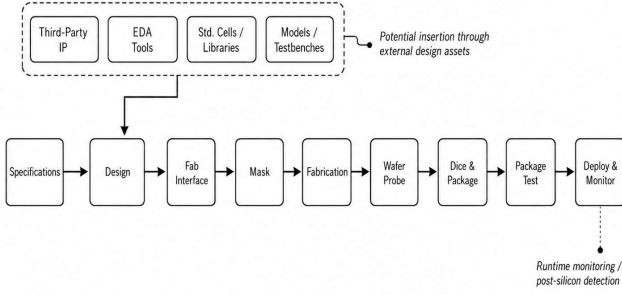


Fig. 1. Hardware Trojan insertion points across the IC/FPGA lifecycle. The figure highlights potential untrusted stages from third-party IP and design tools to fabrication, packaging, deployment, and runtime monitoring, adapted from prior Hardware Trojan and FPGA trust studies [1], [11].

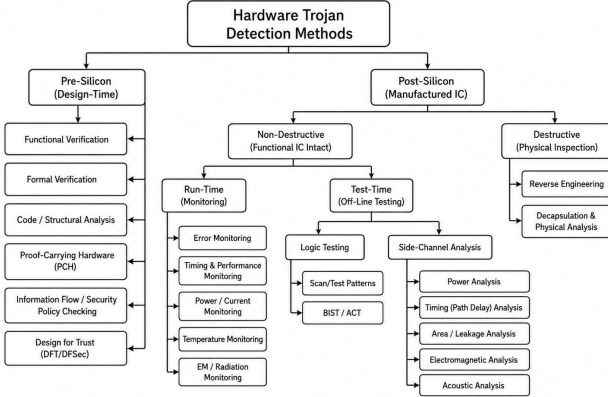


Fig. 2. Taxonomy of Hardware Trojan detection methods. The figure summarizes pre-silicon and post-silicon detection approaches and positions the proposed work under non-destructive runtime side-channel monitoring using embedded digital activity sensors [1], [11], [14].

In cryptographic systems, information-leakage Trojans are especially severe because exposure of the secret key can compromise all protected data. Performance-degradation Trojans are also important because they may induce intermittent faults or reliability failures during field operation.

Fig. 1 shows the assumed Trojan insertion points across the IC/FPGA lifecycle. External design assets such as third-party IP, EDA tools, standard-cell or FPGA libraries, models, and testbenches may influence the design stage. The proposed work focuses on detecting malicious effects during runtime execution.

B. Hardware Trojan Detection Methods

Hardware Trojan detection methods can be broadly categorized into pre-silicon and post-silicon approaches. Pre-silicon methods include formal verification, functional verification, structural analysis, code analysis, and information-flow checking. Post-silicon methods include destructive inspection, functional testing, side-channel analysis, and runtime monitoring [1], [11], [14].

Fig. 2 places the proposed work in the non-destructive runtime side-channel monitoring category. Unlike destructive reverse engineering, the design remains operational. Unlike

external power analysis, the power side-channel proxy is measured internally using digital toggle counters.

C. Side-Channel and Runtime Monitoring

Power-supply transient analysis, IC fingerprinting, path-delay fingerprinting, and ring oscillator networks have shown that malicious modifications can be detected through side-channel deviations [2]–[5], [7], [8]. Machine-learning-based methods have also been explored using side-channel traces, netlist features, LUT-level features, and runtime anomaly models [12], [14], [15], [17], [18].

The proposed method differs from external side-channel methods by embedding the measurement mechanism directly into the AES core. It also differs from static netlist-based detection because it observes the circuit during actual encryption transactions.

III. TOGGLE ACTIVITY AS A SIDE-CHANNEL PROXY

The motivation for toggle-counter monitoring comes from CMOS power behavior. Total power can be expressed as

$$P_{\text{total}} = P_{\text{static}} + P_{\text{dynamic}}. \quad (1)$$

The dynamic switching component is commonly approximated as

$$P_{\text{dynamic}} = \alpha C_L V_{DD}^2 f, \quad (2)$$

where α is the switching activity factor, C_L is the effective load capacitance, V_{DD} is the supply voltage, and f is the clock frequency [22], [23].

In a fixed FPGA implementation operating at a fixed clock and voltage, changes in switching activity become an important contributor to dynamic power variation. Therefore, measuring internal toggle activity provides a digital proxy for power side-channel behavior. Prior power and ring-oscillator-based detection works similarly exploit the relationship between Trojan activity, switching noise, and side-channel deviation [3], [7], [9], [10].

The proposed framework does not claim to replace physical power measurement. Instead, it provides an embedded digital side-channel proxy that is easy to extract during runtime FPGA operation.

IV. PROPOSED FRAMEWORK

A. System Overview

Fig. 3 shows the proposed overall AES FPGA system architecture, which consists of three major blocks: the UART interface, the AES-128 encryption module, and the proposed activity-counter module. The plaintext is transmitted from the host computer to the FPGA through the UART interface. The received data is processed by the AES-128 encryption module to generate the corresponding ciphertext. At the same time, the proposed activity-counter module extracts switching-activity information during the encryption process for analysis and hardware evaluation.

Fig. 4 illustrates the experimental hardware setup used for AES-128 implementation and testing. The Basys 3 Artix-7 FPGA board is connected to a host computer through a UART

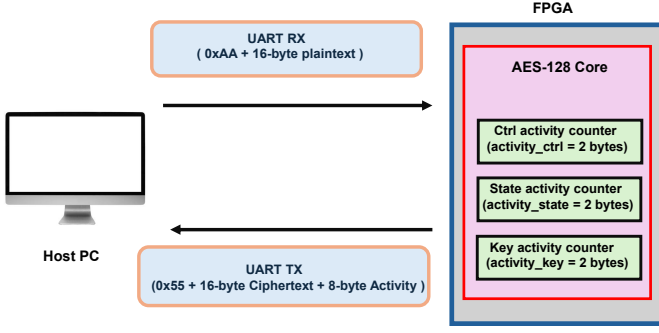


Fig. 3. AES FPGA system architecture showing UART-based plaintext transfer, AES-128 encryption, and activity-counter feature extraction.



Fig. 4. Experimental hardware setup with the Basys 3 Artix-7 FPGA connected to a host computer for UART-based AES testing and activity-data collection.

interface. UART is used for serial communication between the FPGA board and the computer for plaintext transmission and ciphertext reception [24]. It is worth noting that no mixed-signal oscilloscope, logic analyzer, ChipWhisperer platform, current probe, or differential probe is required in the proposed setup for side-channel-related activity monitoring [25].

The FPGA design contains UART receiver/transmitter modules, an AES-128 encryption core, embedded activity sensors, activity output registers, and a host-side Python automation flow. The host validates the FPGA ciphertext against a software AES reference and logs activity features for analysis.

B. Embedded Toggle-Counter Sensor

For a monitored vector $x[t]$, the proposed toggle-counter sensor follows the switching-activity principle commonly used in side-channel power analysis, where data-dependent bit transitions contribute to observable power variations [26], [27]. Such transition-based activity modeling is also consistent with transition-density analysis used for estimating activity in digital circuits [28]. The transition vector is computed as

$$\Delta x[t] = x[t] \oplus x[t-1]. \quad (3)$$

The per-cycle toggle count is

$$a[t] = \text{popcount}(\Delta x[t]). \quad (4)$$

For one AES encryption transaction, the accumulated activity is

$$A = \sum_{t=t_s}^{t_e} a[t], \quad (5)$$

where t_s and t_e are the start and end cycles of the monitored encryption window.

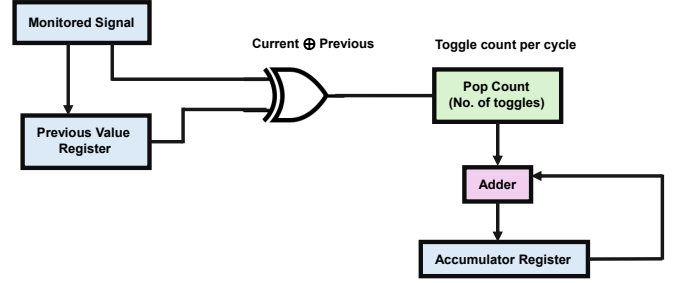


Fig. 5. Per-cycle operation of the embedded toggle-activity detector. The monitored signal is compared with its previous value, bit transitions are counted, and the result is accumulated as an activity fingerprint.

Fig. 5 shows the sensor pipeline. The previous-value register stores the monitored signal from the previous cycle. XOR identifies the toggled bits. Popcount counts the number of toggles, and an accumulator stores the transaction-level activity value.

C. Monitored AES Features

The framework collects four activity features:

- **State activity:** switching activity in the AES internal state path.
- **Key activity:** switching activity in the AES round-key/key-schedule path.
- **Control activity:** switching activity in AES control and FSM-related signals.
- **Total activity:** aggregate activity proxy combining monitored switching behavior.

The control-path feature is particularly important because the clean AES controller follows a deterministic sequence for every encryption transaction. Therefore, Trojan-induced control deviations become visible as control-path activity spikes.

V. TROJAN BENCHMARKS

The final evaluation uses two Trojan families.

A. Performance Degradation Trojan

The Performance Degradation Trojan emulates runtime stress and reliability degradation. It uses a sequential time-bomb trigger implemented as a small FSM that monitors rare plaintext-byte patterns across encryption transactions. Once the programmed plaintext sequence is observed, the trigger

activates a burst payload that increases internal switching activity and stresses the AES datapath/control logic. The Trojan is evaluated in two variants: Case 1 is a stealthier variant with smaller activity impact, while Case 2 is a stronger variant with higher switching impact. In both cases, the rare activation produces sparse runtime anomalies and can lead to intermittent ciphertext mismatches due to stress-induced timing-margin reduction.

B. Information Leakage Trojan

The Information Leakage Trojan represents a confidentiality attack. It is inspired by AES-T1000-style plaintext-triggered key leakage. The trigger plaintext is

$$00112233445566778899AABBCCDDEEFF. \quad (6)$$

When triggered, the Trojan modifies the ciphertext path and reveals the fixed AES key:

$$000102030405060708090A0B0C0D0E0F. \quad (7)$$

This benchmark demonstrates both functional security impact and detectable runtime activity deviation.

VI. DETECTION METHOD

The detector uses a Trojan-free AES baseline. For each activity feature f , the baseline mean μ_f and standard deviation σ_f are computed. The 3-sigma threshold is

$$T_f = \mu_f + 3\sigma_f. \quad (8)$$

A transaction is marked anomalous when

$$f_i > T_f. \quad (9)$$

For the control counter, the clean AES baseline is deterministic:

$$\mu_{\text{ctrl}} = 21, \quad \sigma_{\text{ctrl}} = 0. \quad (10)$$

Therefore, the control-path threshold is

$$T_{\text{ctrl}} = 21. \quad (11)$$

For the Information Leakage Trojan, event-level trigger detection is performed using

$$\text{Ctrl} \geq 24. \quad (12)$$

VII. EXPERIMENTAL EVALUATION

A. Baseline Characterization

Table I shows the Trojan-free AES baseline statistics. The total and state activities vary naturally with random plaintexts. The key activity remains fixed because the AES key is constant. The control activity remains fixed at 21 for all clean samples.

B. Full Control-Path Activity Trace

Fig. 6 shows the full control-path activity trace. The clean AES baseline remains flat at 21. Performance Degradation Trojan Case 1 produces sparse spikes up to 39, while Case 2 produces stronger spikes up to 57. The Information Leakage Trojan produces trigger-event spikes up to 24.

TABLE I
TROJAN-FREE AES BASELINE STATISTICS

Feature	Mean	Std. Dev.	3 σ Threshold
Total	1380.2029	17.7592	1433.4804
State	672.2029	17.7592	725.4804
Key	687.0000	0.0000	687.0000
Ctrl	21.0000	0.0000	21.0000

TABLE II
SAMPLES ABOVE BASELINE-DERIVED THRESHOLDS

Dataset	N	Total	State	Key	Ctrl	Ctrl Rate
Baseline	10000	14	14	0	0	0.00%
Perf. Deg. C1	10000	20	16	0	13	0.13%
Perf. Deg. C2	10000	16	16	0	11	0.11%
Info. Leakage	1000	8	6	0	4	0.40%

TABLE III
MAXIMUM DEVIATIONS FROM TROJAN-FREE BASELINE MEAN

Dataset	Total	State	Key	Ctrl
Performance Degradation Case 1	+72.7971	+72.7971	0	+18
Performance Degradation Case 2	+146.7971	+110.7971	0	+36
Information Leakage	+74.7971	+73.7971	0	+3

TABLE IV
MAXIMUM Z-SCORES FOR TOTAL AND STATE ACTIVITY

Dataset	Max Total z	Max State z
Performance Degradation Case 1	4.10	4.10
Performance Degradation Case 2	8.27	6.24
Information Leakage	4.21	4.16

C. Zoomed State, Total, Key, and Control Evidence

Separate full-run Total and State plots are not included because those features naturally vary with plaintext and become visually less informative when plotted over long runs. Instead, Figs. 7–9 show zoomed baseline-vs-Trojan comparisons of Total, State, Key, and Ctrl activity around representative anomaly events. Baseline and Trojan experiments were collected under the same random plaintext policy; therefore, Total and State overlays are interpreted as supporting activity-pattern evidence, while Ctrl is treated as the primary discriminative feature. These figures preserve the supporting Total/State evidence while keeping the control-path fingerprint central.

D. Statistical Detection Summary

Table II shows the number of threshold-crossing samples. The baseline has zero control-path threshold crossings. The Performance Degradation Trojan cases produce sparse control-path anomalies. The Information Leakage Trojan produces localized trigger-time control-path bursts relative to the clean baseline, with four Ctrl threshold crossings corresponding to the known leakage-trigger events.

Fig. 10 summarizes the statistical evidence and confirms that the control-path fingerprint provides the cleanest separation in the evaluated AES platform.

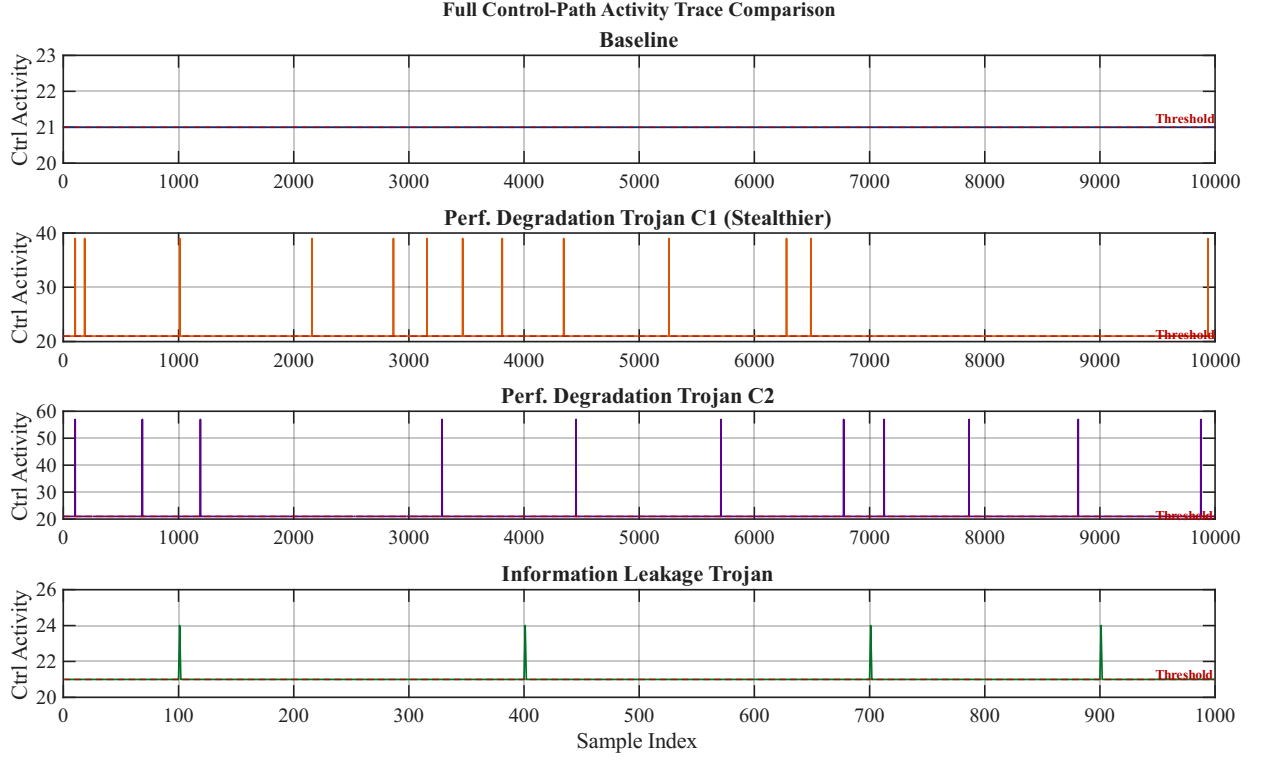


Fig. 6. Full control-path activity trace comparison for the clean AES baseline, Performance Degradation Trojan Case 1, Performance Degradation Trojan Case 2, and Information Leakage Trojan. The baseline remains fixed at 21, while Trojan executions produce threshold-crossing control activity.

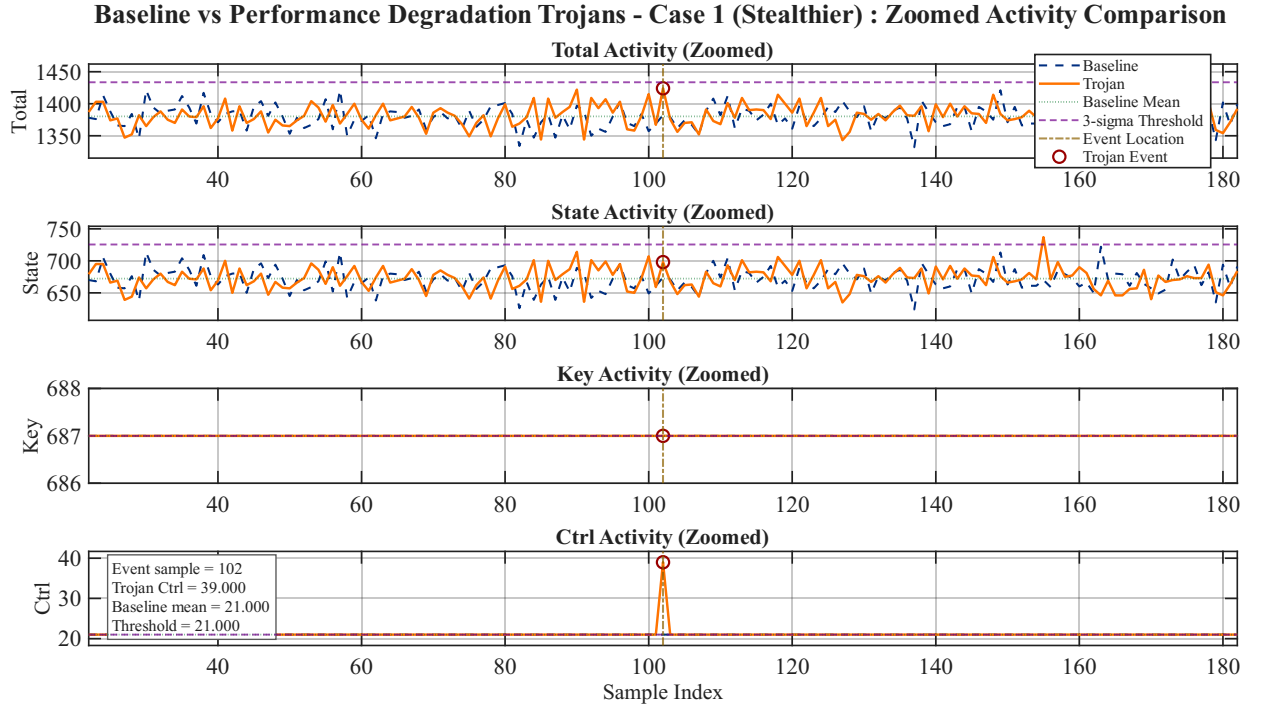


Fig. 7. Baseline vs Performance Degradation Trojan Case 1 zoomed activity comparison. At event sample 102, the control-path activity rises from the baseline value of 21 to 39.

E. Payload Validation

Fig. 11 provides payload-level validation. The Information Leakage Trojan reveals the AES key during trigger activation, while the Performance Degradation Trojan introduces rare

switching-stress events that produce intermittent ciphertext corruption. In the evaluated runs, both Performance Degradation Trojan cases showed an observed ciphertext mismatch rate of approximately 0.1%, confirming that the payload can

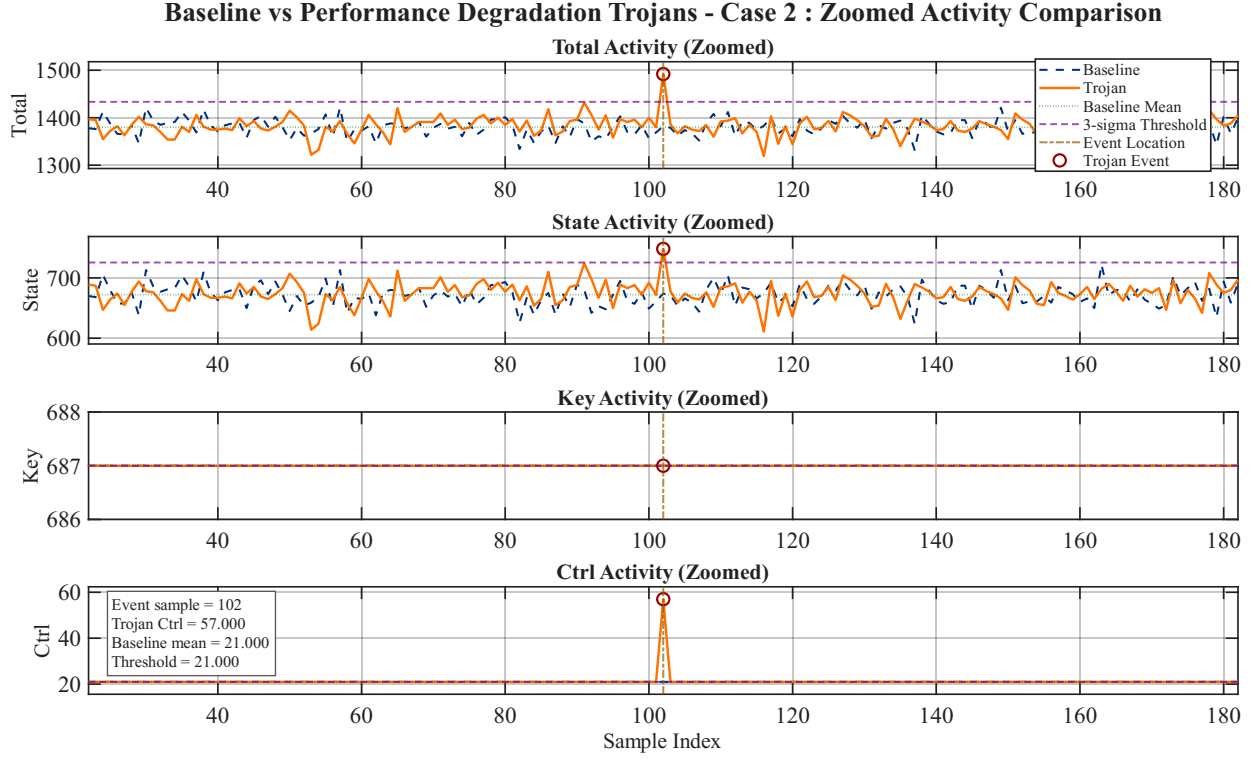


Fig. 8. Baseline vs Performance Degradation Trojan Case 2 zoomed activity comparison. The stronger case reaches a control-path activity of 57 at event sample 102.

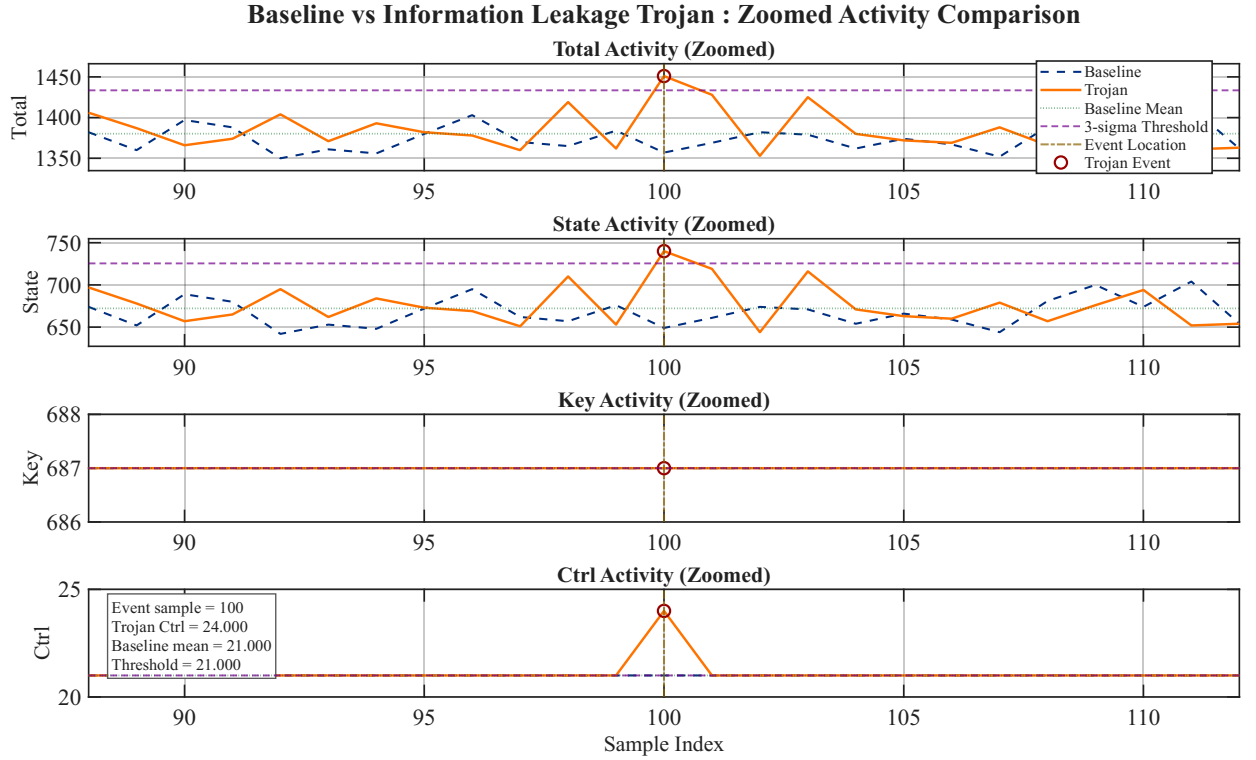


Fig. 9. Baseline vs Information Leakage Trojan zoomed activity comparison. At trigger sample 100, the control-path activity reaches 24, enabling event-level leakage detection.

increase internal switching stress and cause rare functional failures while remaining difficult to detect through ordinary functional testing.

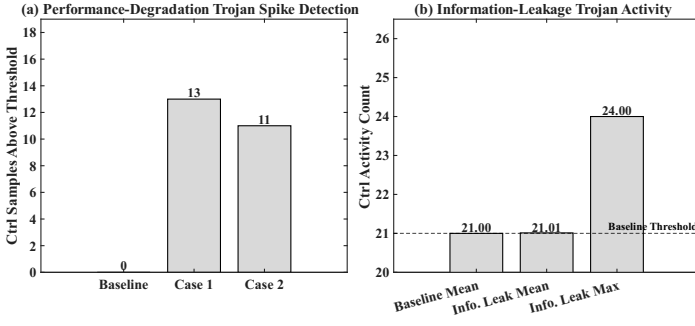
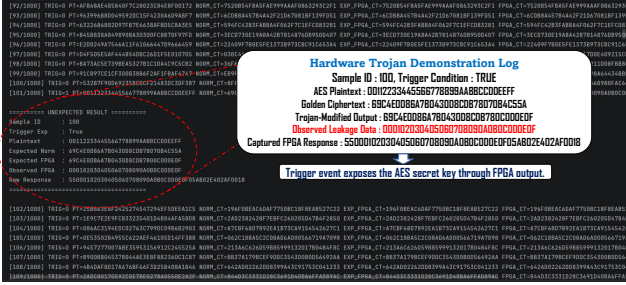


Fig. 10. Baseline-derived statistical anomaly evidence. The control counter produces zero baseline threshold crossings and clear Trojan-event separation.



(a) Information Leakage Trojan payload: trigger event reveals the AES key through the FPGA output.



(b) Performance Degradation Trojan payload: runtime stress produces a ciphertext mismatch.

Fig. 11. Payload-level validation evidence. The Information Leakage Trojan demonstrates key leakage, while the Performance Degradation Trojan demonstrates functional corruption through ciphertext mismatch.

TABLE V
INFORMATION LEAKAGE TROJAN TRIGGER EVENTS

Sample	Total	State	Key	Ctrl
100	≈1451	≈740	687	24
400	≈1451	≈740	687	24
700	≈1451	≈740	687	24
900	≈1451	≈740	687	24

F. Information Leakage Event-Level Metrics

Using $\text{Ctrl} \geq 24$, the trigger events are detected without false positives in the labeled Information Leakage Trojan run, as shown in Table VI.

TABLE VI
INFORMATION LEAKAGE EVENT-LEVEL DETECTION USING $\text{Ctrl} \geq 24$

Metric	Value
True Positives	4
False Positives	0
False Negatives	0
True Negatives	996
Accuracy	100%
Precision	100%
Recall	100%
F1-score	100%

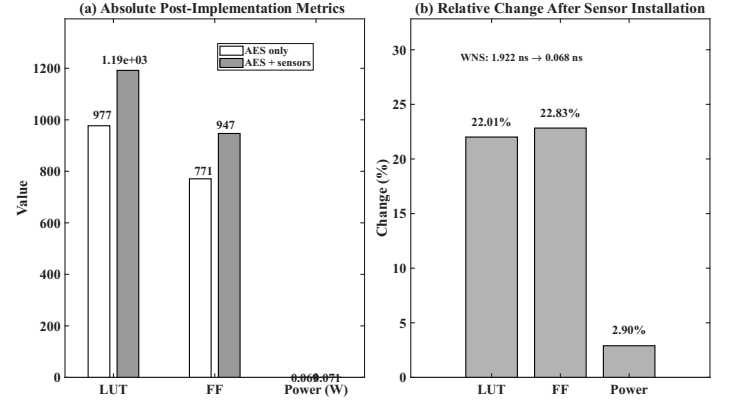


Fig. 12. Implementation overhead of the embedded toggle-counter sensors. The design maintains timing closure with low power overhead.

TABLE VII
POST-IMPLEMENTATION OVERHEAD OF TOGGLE-COUNTER SENSORS

Metric	AES Only	AES + Sensors	Change
LUT	977	1192	+22.01%
FF	771	947	+22.83%
Power	0.069 W	0.071 W	+2.90%
WNS	1.922 ns	0.068 ns	Positive

G. Implementation Overhead

Fig. 12 and Table VII summarize the overhead. The design maintains positive timing slack after sensor integration. The power overhead is only 2.90%, showing that the embedded monitor is practical for FPGA runtime monitoring.

VIII. DISCUSSION

The experiments show that on-chip toggle-counter fingerprinting provides strong runtime evidence for both Trojan families. The clean AES controller follows a fixed execution sequence, producing a stable control activity of 21. Trojan logic perturbs this behavior in different ways depending on the payload.

For the Performance Degradation Trojan, the control-path spikes are sparse and event-like. Case 1 produces a lower-magnitude signature, while Case 2 produces a larger signature. This demonstrates that the proposed detector can distinguish different payload strengths through the activity fingerprint magnitude.

For the Information Leakage Trojan, the payload is security-critical because it reveals the AES key during trigger events.

The activity fingerprint captures this leakage event through a localized control-path rise from the baseline value of 21 to 24. Unlike the Performance Degradation Trojan, its control signature is smaller in magnitude but is aligned with the known leakage-trigger samples.

The total and state counters provide supporting evidence, especially when the Trojan payload causes datapath switching deviation. However, because these counters naturally vary with plaintext, the control counter provides the cleanest statistical separation in the evaluated AES implementation.

IX. CONCLUSION

This paper presented an on-chip toggle-counter fingerprinting framework for real-time Hardware Trojan detection in FPGA-based AES cryptographic cores. The method integrates XOR-popcount-based toggle sensors into AES internal state, key, and control paths and extracts per-transaction activity fingerprints during UART-driven encryption. Evaluation on a Xilinx Artix-7 FPGA shows that the control-path fingerprint remains fixed at 21 for Trojan-free AES executions, while the Performance Degradation Trojan produces sparse spikes up to 39 and 57 for Case 1 and Case 2, respectively. The Information Leakage Trojan produces trigger-event control activity of 24 while leaking the AES key through the ciphertext path. The sensorized AES implementation adds only 2.90% power overhead and maintains timing closure. These results demonstrate that embedded digital toggle-counter fingerprinting is a practical runtime power side-channel proxy for detecting Trojan behavior in FPGA cryptographic systems.

ACKNOWLEDGMENT

The authors would like to thank the Department of Electrical Engineering, Indian Institute of Technology Ropar, for providing the academic and laboratory environment required for this work. The authors also acknowledge the use of AI-assisted tools, including ChatGPT and Claude, for language refinement and formatting support during manuscript preparation. The core research idea, methodology, experimental design, implementation, analysis, and conclusions remain the authors' own.

REFERENCES

- [1] R. S. Chakraborty, S. Narasimhan, and S. Bhunia, "Hardware Trojan: Threats and emerging solutions," in *Proc. IEEE International High Level Design Validation and Test Workshop*, 2009, pp. 166–171.
- [2] D. Agrawal, S. Baktir, D. Karakoyunlu, P. Rohatgi, and B. Sunar, "Trojan detection using IC fingerprinting," in *Proc. IEEE Symposium on Security and Privacy*, 2007, pp. 296–310.
- [3] R. Rad, M. Tehranipoor, and J. Plusquellic, "Sensitivity analysis to hardware Trojans using power supply transient signals," in *Proc. IEEE International Workshop on Hardware-Oriented Security and Trust*, 2008, pp. 3–7.
- [4] Y. Jin and Y. Makris, "Hardware Trojan detection using path delay fingerprint," in *Proc. IEEE International Workshop on Hardware-Oriented Security and Trust*, 2008, pp. 51–57.
- [5] J. Li and J. Lach, "At-speed delay characterization for IC authentication and Trojan Horse detection," in *Proc. IEEE International Workshop on Hardware-Oriented Security and Trust*, 2008, pp. 8–14.
- [6] R. S. Chakraborty, F. Wolff, S. Paul, C. Papachristou, and S. Bhunia, "MERO: A statistical approach for Hardware Trojan detection," in *Cryptographic Hardware and Embedded Systems*, 2009, pp. 396–410.
- [7] X. Zhang and M. Tehranipoor, "RON: An on-chip ring oscillator network for Hardware Trojan detection," in *Proc. Design, Automation and Test in Europe*, 2011.
- [8] A. Ferraiuolo, X. Zhang, and M. Tehranipoor, "Experimental analysis of a ring oscillator network for Hardware Trojan detection in a 90nm ASIC," in *Proc. IEEE/ACM International Conference on Computer-Aided Design*, 2012.
- [9] N. Karimian, F. Tehranipoor, M. T. Rahman, S. Kelly, and D. Forte, "Genetic algorithm for Hardware Trojan detection with ring oscillator network," in *Proc. IEEE International Symposium on Hardware Oriented Security and Trust*, 2015.
- [10] Y. Qin and T. Xia, "Sensitivity analysis of ring oscillator based Hardware Trojan detection," in *Proc. IEEE International Conference on Communication Technology*, 2017.
- [11] S. Mal-Sarkar, R. Karam, S. Narasimhan, A. Ghosh, A. Krishna, and S. Bhunia, "Design and validation for FPGA trust under Hardware Trojan attacks," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 2, no. 3, pp. 186–198, 2016.
- [12] R. Shende and D. D. Ambawade, "A side channel based power analysis technique for Hardware Trojan detection using statistical learning approach," in *Proc. IEEE International Conference*, 2016.
- [13] S. R. Zantout, "Hardware Trojan detection in FPGA through side-channel power analysis and machine learning," M.S. thesis, University of California, Irvine, 2018.
- [14] L. Wu, X. Zhang, S. Wang, and W. Hu, "Hardware Trojan detection at LUT: Where structural features meet behavioral characteristics," in *Proc. IEEE International Symposium on Hardware Oriented Security and Trust*, 2022.
- [15] H. Lashen, L. Alrahis, J. Knechtel, and O. Sinanoglu, "TrojanSAINT: Gate-level netlist sampling-based inductive learning for Hardware Trojan detection," in *Proc. IEEE International Symposium on Circuits and Systems*, 2023.
- [16] A. Sarihi, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Multi-criteria Hardware Trojan detection: A reinforcement learning approach," in *Proc. IEEE Midwest Symposium on Circuits and Systems*, 2023.
- [17] M. R. Gurram, M. Kumar, and F. Amsaad, "Isolation forest based TinyML for detecting Hardware Trojans on FPGA in real time," in *Proc. IEEE Physical Assurance and Inspection of Electronics*, 2024.
- [18] T. Zhang, M. Tehranipoor, and F. Farahmandi, "TrustGuard: Standalone FPGA-based security monitoring through power side-channel," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 32, no. 2, pp. 319–332, 2024.
- [19] N. Chinnasami, R. Stahle-Smith, and R. Karakchi, "ML-enhanced AES anomaly detection for real-time embedded security," arXiv:2507.04197, 2025.
- [20] A. Baltagi, Y. Nasser, and A. Baghdadi, "ML-based Hardware Trojan detection in AI accelerators via power side-channel analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, early access, 2025.
- [21] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Unleashing GHOST: An LLM-powered framework for automated Hardware Trojan design," arXiv:2412.02816, 2024.
- [22] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [23] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.
- [24] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2006.
- [25] C. O'Flynn and Z. D. Chen, "ChipWhisperer: An open-source platform for hardware embedded security research," in *Proc. Int. Workshop Constructive Side-Channel Analysis and Secure Design (COSADE)*, Paris, France, 2014, pp. 243–260.
- [26] E. Brier, C. Clavier, and F. Olivier, "Correlation power analysis with a leakage model," in *Proc. Int. Workshop Cryptographic Hardware and Embedded Systems (CHES)*, Cambridge, MA, USA, 2004, pp. 16–29.
- [27] S. Mangard, E. Oswald, and T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. New York, NY, USA: Springer, 2007.
- [28] F. N. Najm, "Transition density: A new measure of activity in digital circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 12, no. 2, pp. 310–323, Feb. 1993, doi: 10.1109/43.205010.